

Detecção Automática de Quedas

Caio Passos Torkst Ferreira

14 de outubro de 2025

Introdução

Metodologia

Resultados

Conclusão e Trabalhos Futuros

Introdução

- Quedas são eventos críticos em ambientes militares.
- Detecção automática é vital para resposta rápida.
- Machine Learning pode capturar padrões em sensores corporais.
- Projeto baseado no artigo:
 - *A Machine Learning Approach to Automatic Fall Detection of Soldiers*
 - <https://arxiv.org/abs/2501.15655v2>

- Reimplementação em PyTorch com melhorias:
 - Inclusão de LSTM, K-Fold CV, Optuna (bayesiano), Early Stopping
- Pipeline automatizado: `all.sh`
- Dados inerciais do **peito** (acel. + giro, 3 eixos $\rightarrow$ shape: (1020, 6))
- Total de amostras: **6038**, Label: `binary_two`
- Cenário: `Sc_4_T` (tempo)

- 15 voluntários com:
 - 1 smartphone (peito), 2 smartwatches (pulsos)
- Sensores:
 - Aceleração linear (acc) e angular (gyr) em x, y, z
- Amostragem média:
 - Smartphone: 221 Hz (acc), 219 Hz (gyr)
 - Smartwatches: 93 Hz (acc/gyr)

Objetivos do Projeto

- Construir pipeline de detecção de quedas com MLP, CNN1D e LSTM
- Validar com K-Fold, Otimização bayesiana (Optuna) e Early Stopping
- Avaliar com: **MCC, F1, Specificity, Accuracy**
- Estabilidade: 20 execuções por arquitetura
- Interpretabilidade com SHAP e permutação
- Estudar impacto do volume de dados nas curvas de aprendizado

Métricas e Justificativa da Abordagem

- **F1-Score:** Média harmônica entre precisão e revocação — boa com dados equilibrados.
- **MCC:** Considera TP, TN, FP, FN — ideal para dados desbalanceados (1:8).
- Abordagem visa:
 - Robustez e confiabilidade em cenários reais
 - Portabilidade para outros sensores/posições
 - Explicabilidade em contextos críticos

Metodologia

Pipeline e Fluxo do Experimento

- Automação via `all.sh`; etapas principais:
 1. Geração do dataset: `chest/Sc_4_T`
 2. Otimização de hiperparâmetros com **Optuna** (5-Fold CV, MCC)
 3. Treinamento final com os melhores parâmetros
 4. Avaliação estatística, interpretabilidade (SHAP, Permutation)
- Cada modelo é treinado **20 vezes** para avaliar estabilidade.

- **MLP**: 1–3 camadas densas, 32–512 neurônios, dropout [0.0–0.5]
- **CNN1D**: 1–3 convoluções, kernel (3/5/7), filtros (8–64)
- **LSTM**: 1–2 camadas, 16–256 hidden units
- **Learning rate**: $[10^{-4}, 10^{-2}]$ (log)
- Saída comum: função Sigmoid

- **Optuna (TPE)** com 5-Fold CV, métrica alvo: MCC
- Técnicas:
 - Early Stopping : Evita Overfitting
 - Median Pruning : Interrompe trials ruins
- Parâmetros mais relevantes:
 - **Learning rate** → decisivo em todos os modelos
 - CNN1D: kernel e filtros
 - MLP: número de camadas e dropout
 - LSTM: camadas e hidden size

- Após definição dos melhores hiperparâmetros, são treinados **20 modelos independentes**.
- O conjunto de teste (20%) é mantido fixo e só utilizado no final.
- Isso permite medir variância e estabilidade por arquitetura.

- Métricas principais: **MCC**, **F1-score**, **Accuracy**, **Specificity**, **Sensitivity**
- Análises:
 - Curvas de loss e aprendizado
 - Boxplots de estabilidade
 - Interpretação via SHAP e Permutation

- Técnicas aplicadas:
 - **SHAP**: impacto local de cada feature
 - **Permutation Importance**: impacto global
- Objetivos:
 - Entender decisões do modelo
 - Destacar features mais relevantes por arquitetura

Parâmetros gerais:

- Dropout: $[0.0, 0.5]$
- Learning rate: $[10^{-4}, 10^{-2}]$ (log)

Arquitetura	Camadas	Parâmetros específicos
MLP	1–3 densas	32–512 neurônios por camada
CNN1D	1–3 convoluções	Kernel: 3/5/7, Filtros: 8–64
LSTM	1–2 camadas	Hidden size: 16–256

Pipeline do Experimento

1. Comandos CLI

- Escolher Arquitetura (CNN, LSTM, MLP)

2. Divisão dos Dados

- 80% treino / 20% teste

3. Otimização (Optuna)

- 5-Fold CV e MCC médio

4. Treinamento Final

- 20 modelos com os melhores hiperparâmetros (por arquitetura)
- Avaliação no conjunto de teste

5. Análise

- Boxplots, curvas, classification report
- SHAP e Permutation Importance

Resultados

Melhores Hiperparâmetros

- **CNN1D**

- **Parâmetros:**

- Dropout: 0.1
 - Learning Rate: 0.00059
 - Threshold: 0.5
 - Filtro: 59, Kernel: 6
 - Camadas Convolucionais: 4
 - Camadas Densas: 1 com 300

- **MLP**

- **Parâmetros:**

- Dropout: 0.3
 - Learning Rate: 0.00128
 - Threshold: 0.6
 - Camadas Densas: 2 com 643

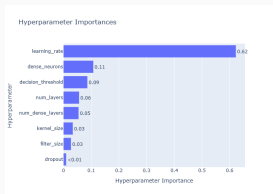
- **LSTM**

- **Parâmetros:**

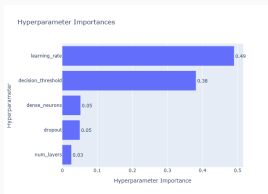
- Dropout: 0.1
 - Learning Rate: 0.00143
 - Threshold: 0.7
 - Camadas LSTM: 1 com 87

Importância dos Hiperparâmetros (Optuna)

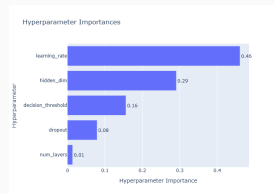
- **Learning rate** é o hiperparâmetro mais relevante em todas as arquiteturas.
- **CNN1D:** kernel size e número de filtros também têm impacto importante.
- **MLP:** número de camadas e taxa de dropout afetam o desempenho.
- **LSTM:** sensível ao número de camadas e unidades escondidas.



CNN1D



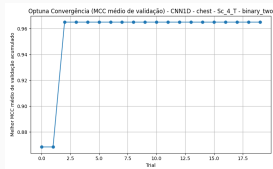
MLP



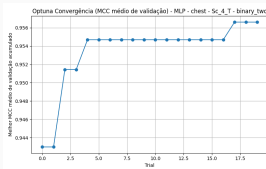
LSTM

Convergência do Optuna (Histórico de Trials)

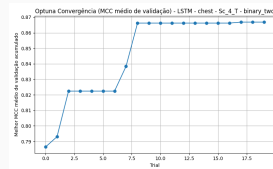
- Atenção ao dataset!
- **CNN1D** mostrou convergência mais estável e suave.
- **MLP** atingiu ótimos resultados rapidamente com menos oscilações.
- **LSTM** apresentou maior variabilidade entre trials, refletindo instabilidade.



CNN1D



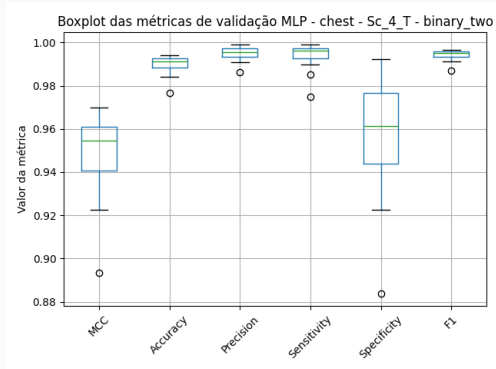
MLP



LSTM

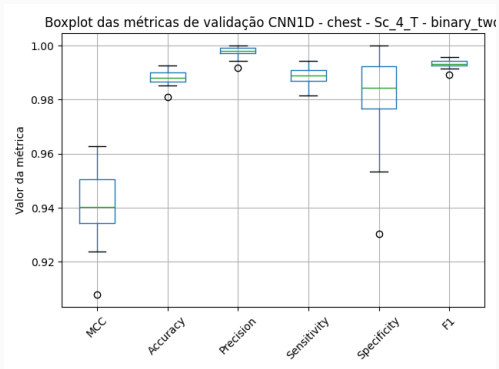
- Cada arquitetura foi treinada 20 vezes para avaliar a variabilidade.
- A CNN1D obteve o melhor desempenho médio e menor variância.
- O MLP mostrou resultados estáveis e competitivos.
- O LSTM apresentou maior variabilidade e inconsistência nos resultados.

Boxplot das Métricas – MLP



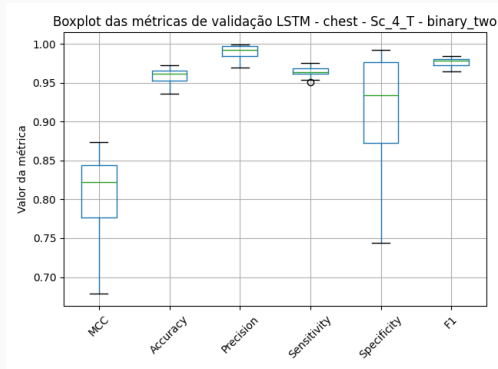
MLP

Boxplot das Métricas – CNN1D



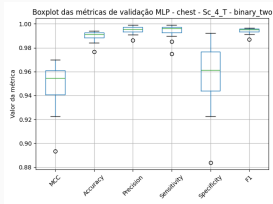
CNN1D

Boxplot das Métricas – LSTM

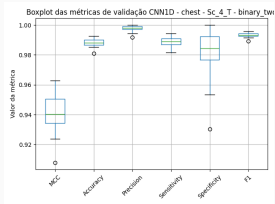


LSTM

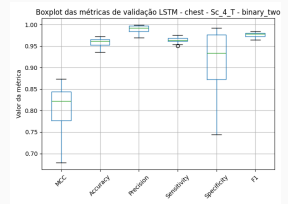
Boxplot das Métricas



MLP



CNN1D



LSTM

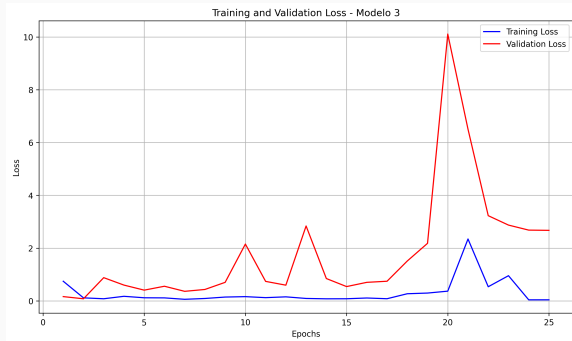
Médias e Desvios Padrão das Métricas

Tabela 1: Resultados dos modelos na posição **chest**, cenário **Sc_4_T**, com rotulagem **binária (binary_two)**. Os valores estão no formato *média $\pm$ desvio padrão*.

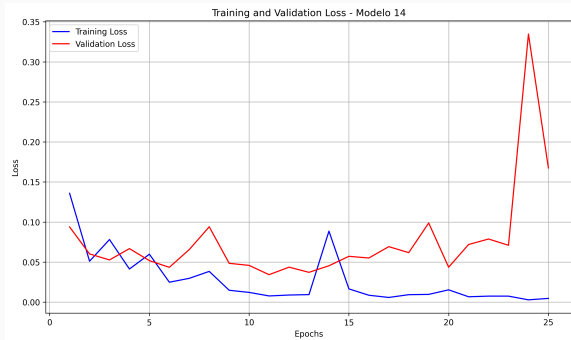
Modelo	MCC	Acurácia	Precisão	Recall	TNR
CNN1D	0.94 $\pm$ 0.01	0.99 $\pm$ 0.00	1.00 $\pm$ 0.00	0.99 $\pm$ 0.00	0.98 $\pm$ 0.02
LSTM	0.81 $\pm$ 0.06	0.96 $\pm$ 0.01	0.99 $\pm$ 0.01	0.96 $\pm$ 0.01	0.91 $\pm$ 0.08
MLP	0.95 $\pm$ 0.02	0.99 $\pm$ 0.00	0.99 $\pm$ 0.00	0.99 $\pm$ 0.01	0.96 $\pm$ 0.03

- CNN1D e MLP apresentam alta acurácia e estabilidade.
- LSTM demonstra maior variabilidade e desempenho inferior, principalmente em MCC e especificidade.

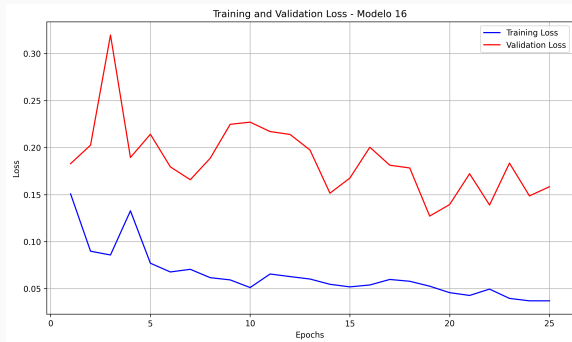
Curva de Perda — MLP



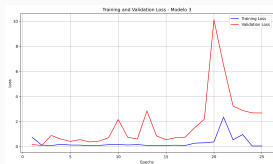
Curva de Perda — CNN1D



Curva de Perda — LSTM



Curvas de Perda



MLP



CNN1D



LSTM

Comparação - Curva de Loss

- **CNN1D**

- Treinamento com boa convergência (perda ≈ 0.01 no final).
- Validação estável até ~ 20 épocas, mas explosão de perda após ~ 22 épocas (overfitting evidente).

- **LSTM**

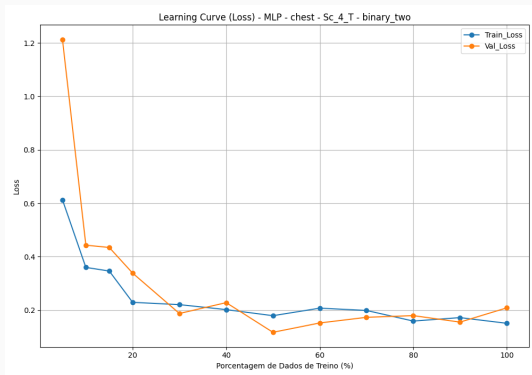
- Perda de validação consistentemente mais alta que a de treinamento (gap significativo).
- Overfitting desde o início (modelo memoriza rapidamente os dados de treino).
- Possível necessidade de mais dados ou maior regularização.

- **MLP**

- Treinamento estável, mas perda de validação extremamente volátil.
- Picos abruptos (até ~ 10 de perda) indicam instabilidade do otimizador ou taxa de aprendizado muito alta.

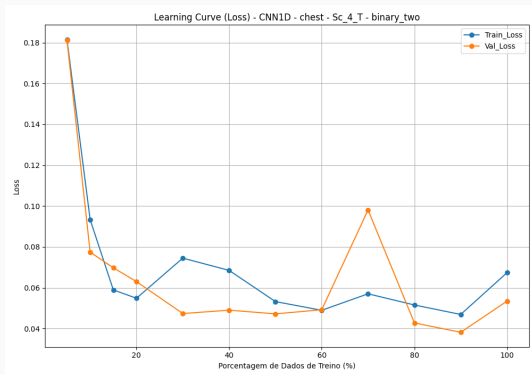
- Avalia impacto da quantidade de dados no desempenho.
- CNN1D: melhora significativa com mais dados — potencial de escalabilidade.
- MLP: robusto mesmo com poucos dados.
- LSTM: resultados imprevisíveis, sem tendência clara.

Curva de Aprendizizado – MLP



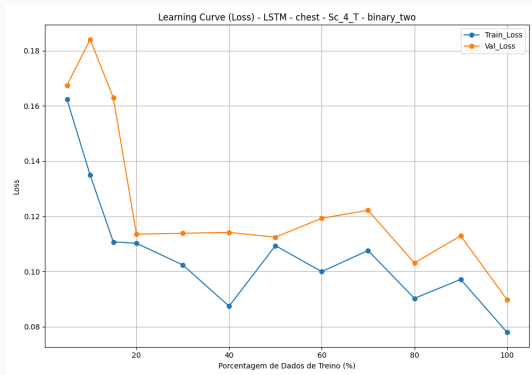
MLP

Curva de Aprendizado – CNN1D



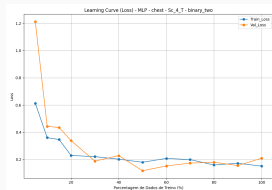
CNN1D

Curva de Aprendizado – LSTM

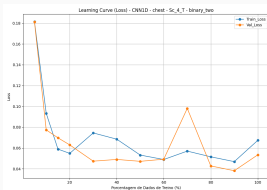


LSTM

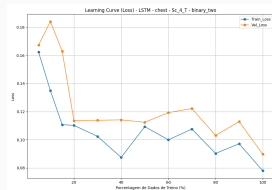
Curvas de Aprendizado (Loss)



MLP



CNN1D



LSTM

Learning Curve (Loss)

- **CNN1D:**

- Perda de validação próxima à de treino na maioria dos pontos.
- Pequenas oscilações indicam certa instabilidade, mas não há sinais claros de overfitting.

- **LSTM:**

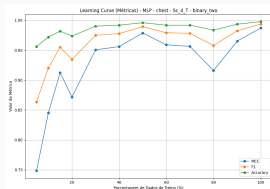
- Val Loss consistentemente acima do Train Loss.
- Indício de overfitting leve a moderado, especialmente nas maiores porcentagens de dados.

- **MLP:**

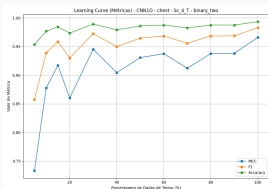
- Gap considerável entre Train e Val Loss nas menores quantidades de dados.
- Com mais dados, as perdas convergem, reduzindo o overfitting.

Conclusão: Todos os modelos se beneficiam do aumento de dados. LSTM demonstra maior tendência a overfitting. CNN e MLP apresentam boa generalização na maior parte dos cenários.

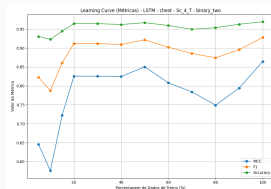
Curvas de Aprendizado (Métricas)



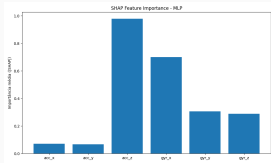
MLP



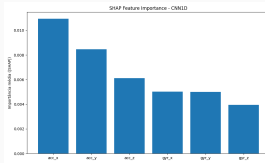
CNN1D



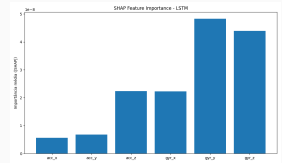
LSTM



MLP



CNN1D



LSTM

Análise de Importância de Atributos (SHAP)

Objetivo: Comparar quais sensores (acelerômetro vs giroscópio) foram mais relevantes para cada modelo.

- **CNN1D:**

- Acelerômetro dominou a decisão (`acc_x` e `acc_y` com maior importância).
- Giroscópio teve contribuição secundária, relativamente equilibrada entre os eixos.

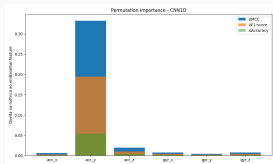
- **MLP:**

- Forte dependência de um único atributo (`acc_z`), seguido de `gyr_x`.
- Indica sensibilidade a padrões específicos do eixo vertical do acelerômetro.

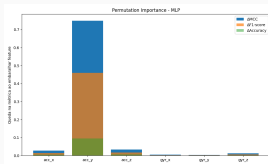
- **LSTM:**

- Giroscópio foi mais relevante (`gyr_y` e `gyr_z` com maiores valores SHAP).
- Acelerômetro teve menor impacto, sugerindo que o LSTM capturou melhor dinâmicas angulares no tempo.

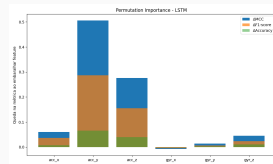
Permutation Importance



CNN1D



MLP



LSTM

Conclusão e Trabalhos Futuros

- A abordagem proposta mostrou que redes neurais — mesmo arquiteturas simples como **MLPs** — podem ser eficazes.
- O modelo **CNN1D** e **MLP** apresentou o melhor desempenho geral:
 - Alta acurácia, F1-score, MCC e baixa variância.
 - Boa interpretabilidade e escalabilidade.
- O **LSTM**, apesar de capturar bem o longo prazo, mostrou instabilidade mesmo após otimização com Optuna. (*)

Limitações do Estudo

- Avaliação restrita ao cenário Sc_4_T com sensores na posição chest.
- Erro no "shuffle"
- Não foi realizado a engenharia de features.
- Interpretação dos modelos feita apenas com dados offline (estáticos).
- Poucos dados.

- Avaliar outros **dispositivos e posições corporais** (ex: lado esquerdo e direito).
- Avaliar ganho com as features em domínio da frequência (Fourier).
- Fazer corretamente a engenharia de features para treinar os modelos. (magnitude)
- Explicar SHAP temporalmente? ForcePlots, Waterfall, Dependence, etc.
- Explorar **métodos ensemble** e **conformal prediction** para maior confiabilidade.
- Desenvolver uma **versão embarcável ou em tempo real** para aplicação prática.
- Avaliar **arquiteturas híbridas**.

Obrigado!